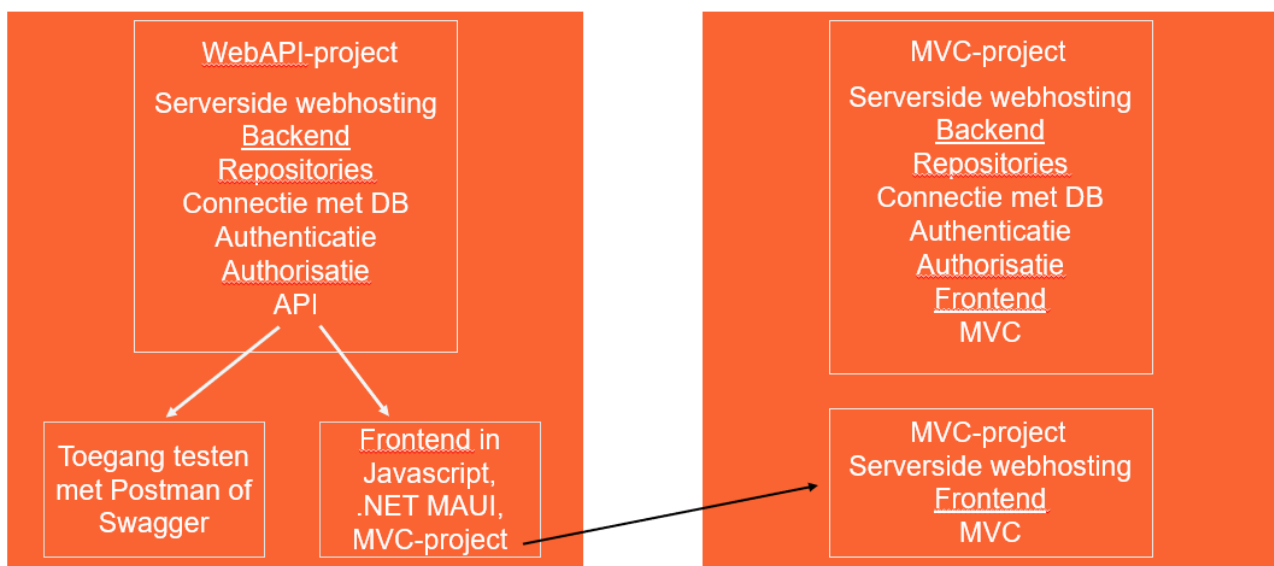


09 MVC deel 1 - Inleiding

📅 09-09-2026 ⌚ Ongeveer 17 minuten 🗨️ Ongeveer 2555 woorden

In de vorige hoofdstukken leerde je een API-backend bouwen om gegevens uit een database te manipuleren en te beveiligen. Je gebruikte Swagger om endpoints aan te spreken. In de volgende hoofdstukken ontwikkel je een UI in een MVC-project. Dat is een webapplicatie die je op twee manieren kunt opbouwen:

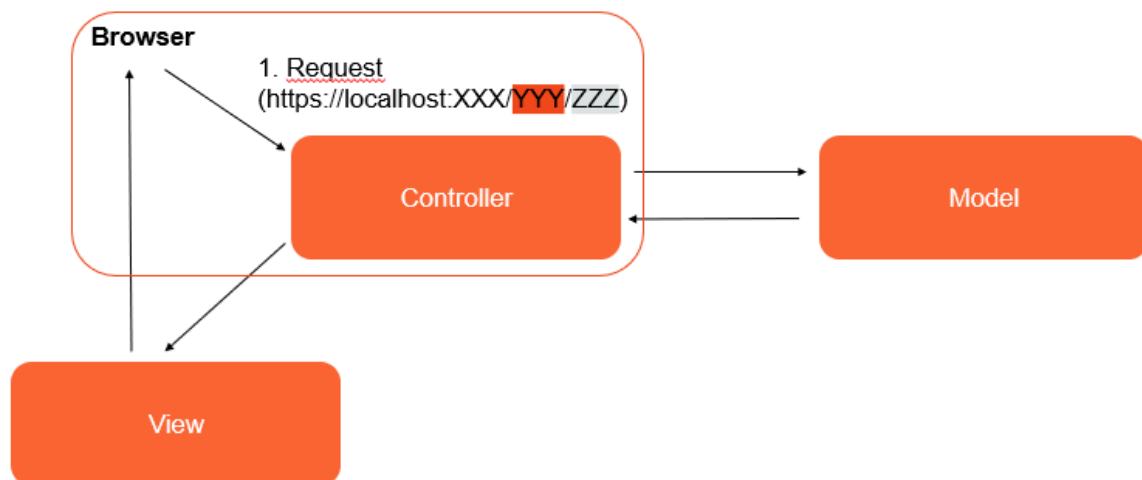
1. Je bouwt een applicatie met enkel de MVC-frontend die, met behulp van HTTP-requests, een WebAPI aanspreekt en zo de gegevens ophaalt voor de View. (Deze methode wordt in deze cursus niet behandeld en is zelfstudie)
2. Of je bouwt een webapplicatie die de MVC-frontend integreert met de backend van de API. De logica van de ApiController komt dan in de Controller van MVC. (Dit wordt de komende drie hoofdstukken uitgewerkt)



MVC-principe

Vorig jaar leerde je in Programming Advanced hoe je een applicatie kunt opdelen in verschillende lagen (MVVM). MVC is een gelijkaardig patroon dat veel gebruikt wordt in webapplicaties. MVC staat voor **Model-View-Controller**. Dit principe vind je ook terug in andere programmeertalen als Django (Python), Laravel (PHP), Spring (Java)...

Overzicht MVC Patroon



- Een **View** bevat net zoals in .NET Maui de logica voor de UI. In MVC gebruik je geen Xaml, maar **html in combinatie met Razor**. Razor is C# code die je tussen je html-code schrijft.
- De **controller** bevat de logica die verantwoordelijk is voor het aanleveren van gegevens aan de **View** en de afhandeling van acties op de View. De Controller neemt de rol van het ViewModel of de ApiController over.
- Het **Model** bevat de klassen met de properties. De methoden om gegevens uit de database te halen en dergelijke zitten in de Controller of een Repository/UnitOfWork.

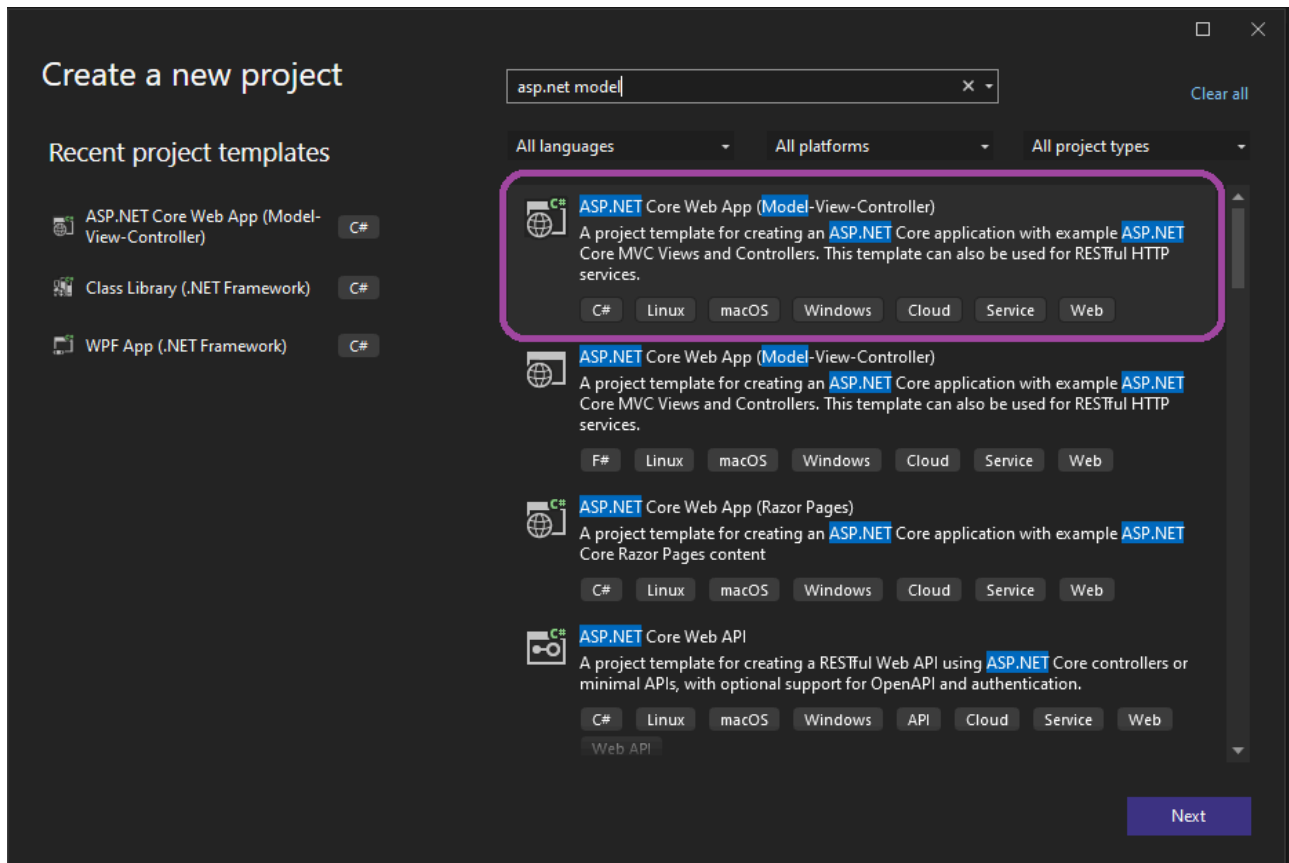
Binnen deze cursus wordt dit MVC-patroon nog uitgebreid met **ViewModels** (Hoofdstuk 10). In MVVM wordt het ViewModel gebruikt als laag tussen Repository, Model en View. Het ViewModel bevat de logica om de gegevens voor de View klaar te zetten. In het MVC-verhaal daarentegen neemt de Controller deze taak over. In MVC wordt het ViewModel gebruikt als doorgeefluik tussen View en Controller. Dit is vergelijkbaar met een DTO voor API's. Als je een View hebt, gebaseerd op één model, heb je geen ViewModel nodig. Meer uitleg volgt.

Voordelen met MVC:

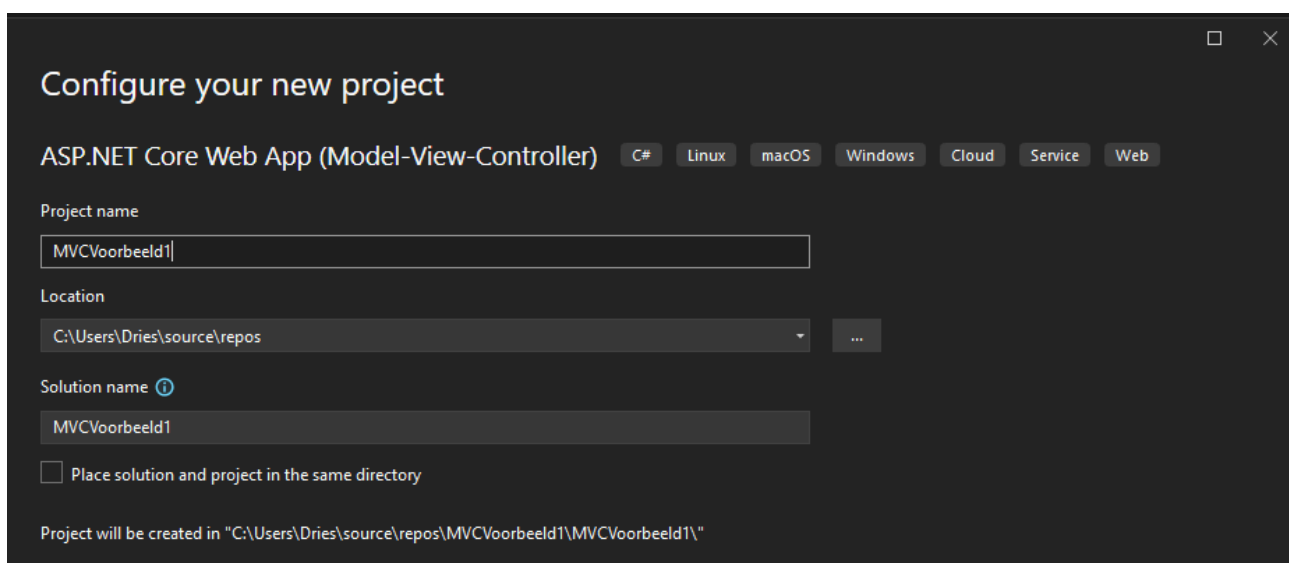
- Testbaar en handig te debuggen
- Beter onderhoudbaar
- Eenvoudiger te updaten
- Separation of Concerns (opsplitsen van je code in lagen zodat elke laag enkel verantwoordelijk is voor zijn deeltje van de applicatie)

MVC-project aanmaken

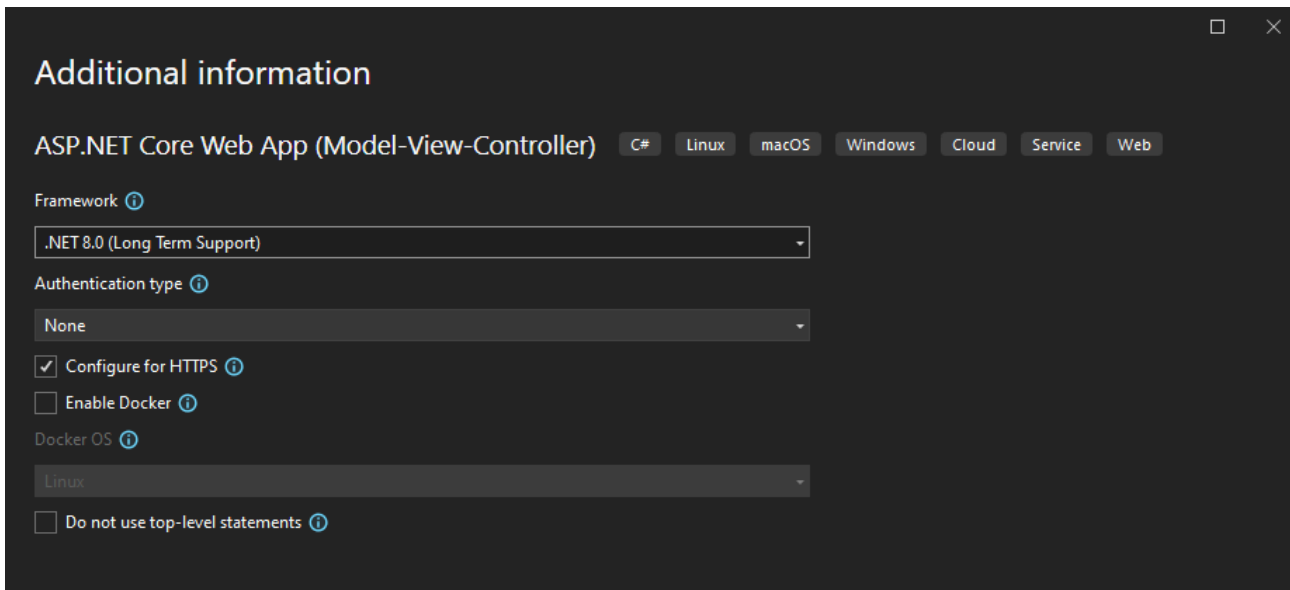
Selecteer ASP.NET Web App (Model-View-Controller) met C#



Stel de naam van het project en solution in.



Kies vervolgens voor de juiste versie die gebruikt wordt binnen deze cursus: asp.net 8.0.



Additional information

ASP.NET Core Web App (Model-View-Controller) C# Linux macOS Windows Cloud Service Web

Framework ⓘ
[.NET 8.0 (Long Term Support)]

Authentication type ⓘ
[None]

☒ Configure for HTTPS ⓘ

☐ Enable Docker ⓘ

Docker OS ⓘ
[Linux]

☐ Do not use top-level statements ⓘ

Met **Authentication Type** ben je reeds bekend van het thema API, je kan dus een **membership systeem** voorzien voor registreren, inloggen of beheren of eventueel rechten te beheren af te dwingen. In dit geval kies 'None' bij Authentication Type.

In een nieuwe project zit een vergelijkbare mappenstructuur als bij WebAPI, maar zonder voorbeeld. Je krijgt een extra map **wwwroot** waar afbeeldingen, CSS, javascript... bewaard worden. Dit is wat gedownload wordt naar je pc als je naar een website surft in een browser. Daarnaast is ook een map **Views** aanwezig waar je de UI gaat opbouwen.

Controller

Een controller handelt **HTTP-verzoeken** (requests) af van de gebruiker. De controller zal dan beslissen welke actie er moet worden uitgevoerd? Moet er data worden opgehaald uit de database, of moeten er records worden toegevoegd, gewijzigd of verwijderd doormiddel van CRUD-operaties? De controller verzamelt alle gegevens en toont daarna de juiste pagina (View) met de juiste inhoud.

Stel een gebruiker surft naar:

- case A: `www.bedrijf.com/Bestelbon` Hier worden alle bestelbonnen opgehaald en getoond op het scherm.
- case B: `/Bestelbon/Bekijken/1875` Hier zal één specifieke bestelbon worden opgevraagd met één bepaalde primaire sleutel. De gebruiker bepaalt deels mee welke output hij wil.

Voor elke functionaliteit van een bepaalde entiteit, maak je een **Action Method** in de Controller van die entiteit.

Een controller erft over van **Controller** ipv **ControllerBase** bij een WebAPI. **ControllerBase** is de basisklasse voor het afhandelen van HTTP-verzoeken en biedt geen afhandeling van weergave-gerelateerde functies van MVC. **Controller** is een uitbreiding van **ControllerBase** en wordt gebruikt voor het afhandelen van zowel API-endpoints als Views.

Routing en namingcovention

Elke Action Method wordt aangesproken via een URL. De opbouw URL wordt bepaald door de controller en de Action Method: Voorbeeld: `https://localhost:XXX/YYY/ZZZ` - XXX bepaalt het poortnummer - YYY bepaalt de Controller - ZZZ bepaalt de Action Method

`https://localhost:5001/product/lijst`

- Naming convention
 - Controller maakt gebruik van suffix 'Controller'
 - Zo weet ons project dat dit een Controller is
 - Voorbeeld: `HelloWorldController`

Default Routing

In `Program.cs` bevindt zich de default route die gebruikt wordt bij het opstarten van je project.



```
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");
```

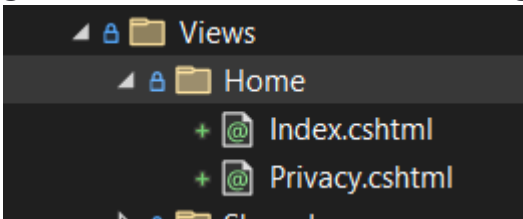
De default waardes worden gehanteerd als er een onderdeel mist in de URL.

- Voor Controller: `HomeController`
- Voor Action Method: `Index`

Er is ook een mogelijkheid voorzien om een Id mee te geven aan de URL.

Action Methode

In de HomeController bevinden zich standaard een aantal Action Methodes zoals `Index` , `Privacy` en `Error` . In onderstaande methode is **Index** niet alleen de naam van de methode maar ook van het endpoint of route. Als resultaat geef je een **View** terug. Je kunt tussen de haakjes de naam van de View opgeven of eventueel parameters meegeven. Als je de naam weglaat, gebruikt hij de View met dezelfde naam als de Action Methode. In dit geval is dat `Index`. De Views worden georganiseerd per Controller.



Als je de applicatie start, surf je naar `http://localhost:poortnr/Home/Index`

```
1 public IActionResult Index(){  
2     return View();  
3 }
```

CS

Je kunt een route van een Action Methode ook benoemen, in een kenmerk boven de Action Methode. `http://localhost:poortnr/Home/Startpagina`

```
1 [Route("StartPagina")]  
2 public IActionResult Index(){  
3     return View();  
4 }
```

CS

Route parameters

Een route kan één of meerdere parameters bevatten, dit is vaak een id, zodat je de details kunt opvragen en tonen. Parameters worden opgevangen in de Action Method indien correct ingevuld.

Dit is een voorbeeld van een route met één parameters:

```
1 public IActionResult Index(int id){  
2     ViewBag.Id = id; //ViewBag is een container waarmee je gegevens kan  
3     doorgeven aan een View. Deze is slechts geldig tijdens de huidige  
4     request.  
     return View();  
}
```

Of met een expliciete route:

```
1 [Route("home/index/{id?}")]  
2 public IActionResult Index(int id)  
3 {  
4     ViewBag.Id = id;  
5     return View();  
6 }
```

Opmerkingen

- Parameters zijn niet verplicht op voorwaarde dat je **een vraagteken** plaatst bij de parameter naam. **Let op:** indien het vraagteken niet aanwezig is bij de id en je doet een GET-request (bv: /Bestelbon/bekijken/ zonder de id) zal de applicatie crashen bij het opstarten.
- Een andere belangrijke voorwaarde is de volgorde, bij route parameters moet dit overeenkomen met opvangen van de parameters in de action én ook correctmatige instelling in program.cs.

Voorbeeld met meerdere routeparameters:

```
1 [Route("users/{userId}/orders/{orderId}")]  
2 public IActionResult OrderDetails(int userId, int orderId)
```

Query parameters

Query parameters zijn parameters die je op een specifieke wijze via de URL meegeeft. Er is geen configuratie in de default route nodig en de volgorde is niet belangrijk. Alle query parameters zijn optioneel.

Vb URL - **Route** parameters:

/Bestelbon/Bekijken/5

Vb URL - **Query** parameters:

/Bestelbon/Bekijken?id=5

In onderstaande Action Methode geef je een string terug in plaats van een View.

```
1 public string Boodschap(string voornaam, string boodschap)    CS
2 {
3     return $"Boodschap van {voornaam}: {boodschap}";
4 }
```

View

Een **View** toont de User Interface (UI) van de applicatie en is opgebouwd uit HTML, CSS, JS gecombineerd met Razor-code (C# code binnen HTML) om eventuele data, die wordt aan-geleverd via de Controller, te tonen.

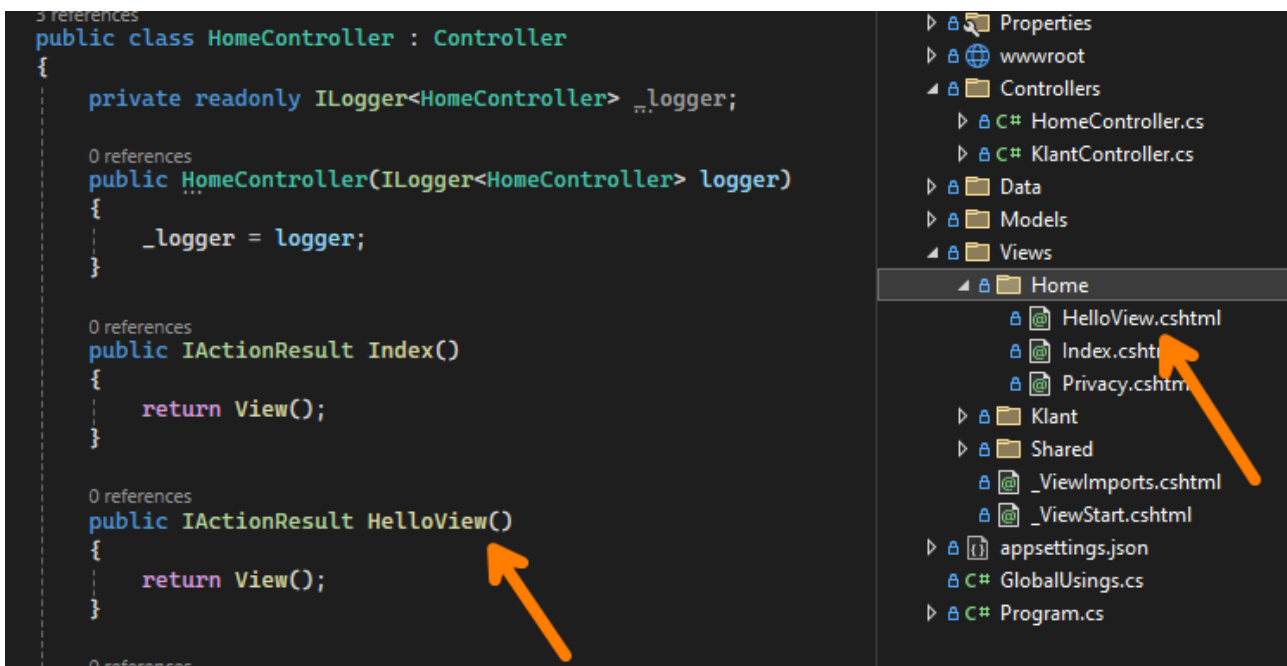
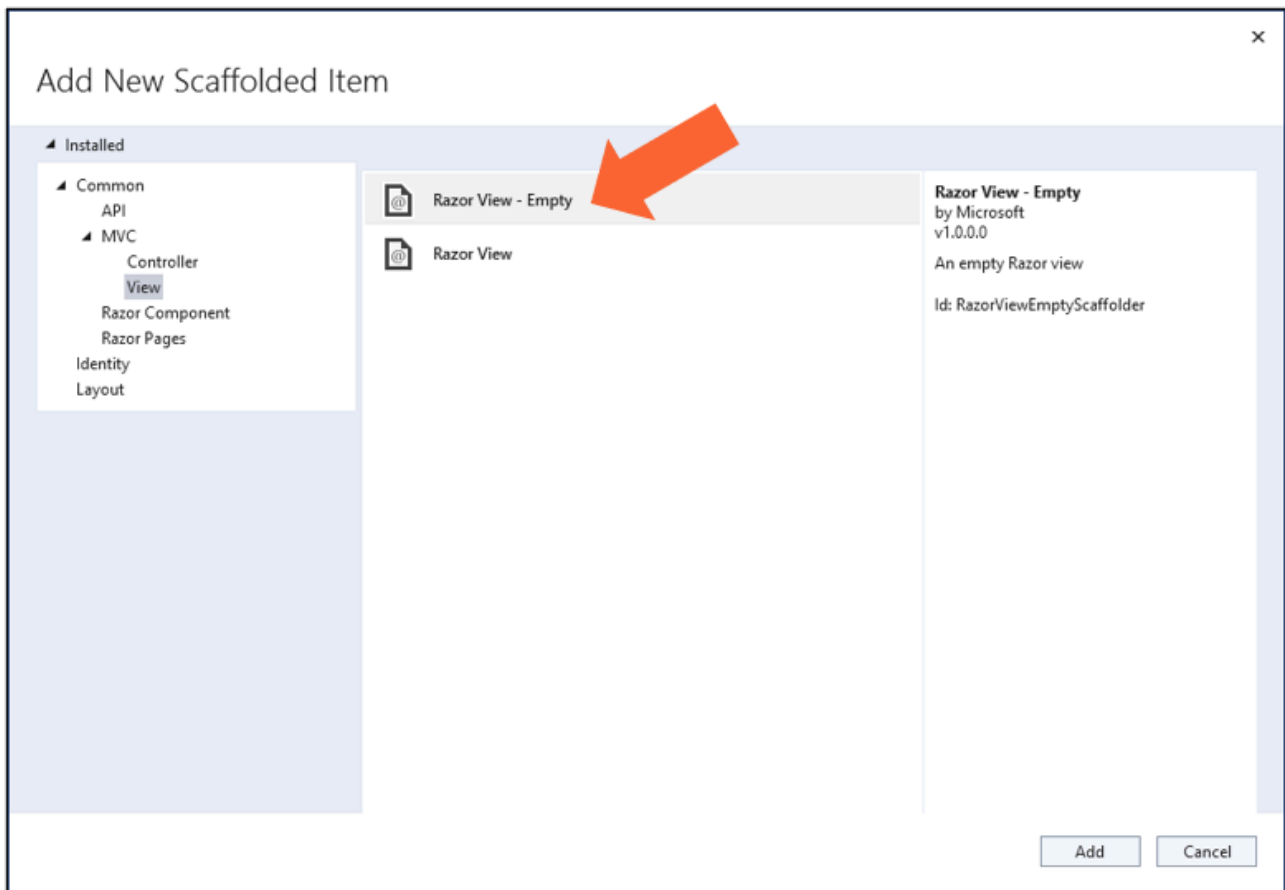
Naming convention:

- Als je een nieuwe Action Methode maakt in je Controller, maak je ook een View aan met dezelfde naam als je Action Methode.
- Je plaatst deze, in de map `views`, in een onderliggende map met de naam van de Controller.

Maak in de HomeController een actie `HelloView`

```
1 public IActionResult HelloView()    CS
2 {
3     return View();
4 }
```

Voeg een nieuwe View toe in de map `views\Home` van het type **Razor View Empty**. Geef als naam **HelloView.cshtml**.



Geef volgende html-code in:

```
1 <h2>Hello View</h1>
2 <p>Dit is mijn HelloView page!!</p>
3 <p style="display: none;">Dit is mijn eerste View - Yes!</p>
```

html

Test uit met de Url: <https://localhost:poortnr/Home/HelloView>

Layout

De opbouw van de website zit standaard geprogrammeerd in

`Views\Shared\_Layout.cshtml` . Dit is de **default** lay-out voor de applicatie en bevat de header, renderbody en footer. Er zijn koppelingen aanwezig naar css en bootstrap en in de footer worden er ook een aantal geautomatiseerde zaken geïnjecteerd, zoals javascript (die nodig is voor de validaties van de formulieren).

Je kunt ook je eigen implementatie(s) maken van de layout. Neem daarvoor een kopie van `_Layout.cshtml` en verander de naam in `Views\_ViewStart.cshtml` . Dan wordt jouw layout gebruikt in plaats van de standaard.

`@RenderBody()` wordt hier gebruikt om **de pagina** (bv. `HelloView.cshtml`) in de layout te laden.

Razor

Razor-code is Server side markup language. Bij een http-verzoek gaat de server de razor-code, die je toevoegt aan je HTML-pagina, uitvoeren en geeft dan als resultaat een webpagina die bestaat uit enkel HTML/CSS/javascript/...

Praktisch

- Blokken Razor-code staat tussen `@{ ... }`
- Expressies (variabelen en code-structuren) starten met `@`
- Commentaar plaats je in `@*{ ... }*`

Je kan ook alles op een lijn schrijven indien dit niet nadelig is voor de efficiëntie.

Voorbeelden

Declaraties variabelen

```
1 //Declaren inhoud
2 @{var mijnVariable = "Hallo wereld!";}
3
4 // Displayen waarde
5 <p>De waarde van mijnVariable is: @mijnVariable</p>
6
7 // Block statement
8 @{
9     var aanspreking = "Welkom op onze site!";
10    var weekdag = DateTime.Now.DaysOfWeek;
11    var bericht = $"{aanspreking} Hier is het {weekdag}";
12 }
```

Controlstructuren

Een controlestructuur (if/else) met modulo (rest staartdeling) om te bepalen of de rest even is of niet.

```
1 @if(value % 2 == 0) {
2     <p>De waarde is even</p>
3 } else {
4     <p>De waarde is oneven</p>
5 }
```

Een controlestructuur (switch) is gelijkaardig, je plaatst het @ voor switch en implementeert deze zoals je normaal bij C# ook deed. De Html kan je gewoon erin plaatsen zoals bij bovenstaande if-structuur, omdat je Razor geactiveerd hebt met @.

Loop

Een For-iteratie

In dit voorbeeld krijgt de View van de controller een variabele *Aantal* mee. Je gaat dan zoveel keer het aantal de stappen afdrukken.

```
1 @for(int i = 0; i < (int)ViewData["Aantal"]; i++) {
2     <li>Stap: @i</li>
3 }
```

Een Foreach-iteratie Dit voorbeeld gaat een tabel tonen met alle producten. Je toont van elk product de id, naam en prijs.

```
1  @foreach (var product in Model)
2  {
3      <tr>
4          <td class="col-md-4">@Html.DisplayFor(modelItem =>
5  product.ProductID) </td>
6          <td class="col-md-4">@Html.DisplayFor(modelItem =>
7  product.Naam) </td>
8          <td class="col-md-4">@Html.DisplayFor(modelItem =>
9  product.Prijs) </td>
10         </tr>
11     }
12 }
```

CS

ViewData - ViewBag

Zowel ViewData als ViewBag worden, in ASP.NET MVC, gebruikt om data door te geven van de controller naar de view. ViewData vereist expliciete sleutels om data op te slaan en op te halen en ViewBag gebruikt dynamische properties, waardoor de syntaxis wat cleaner en meer "C#-achtig" aanvoelt. ViewBag is eigenlijk een wrapper rond ViewData. Het maakt gebruik van dezelfde opslagstructuur onder de motorkap. Wanneer je iets aan ViewBag toevoegt, wordt het intern in ViewData opgeslagen.

Beide worden gebruikt om tijdelijk variabelen door te geven naar de View die geen complex Model vereisen.

ViewData wordt in een MVC-applicatie vaak gebruikt om de titel van de View door te geven. De titel heeft niks met onze model of database data te maken.

```
1  public IActionResult HelloView() {
2      ViewData["Title"] = "HelloView";
3      return View();
4  }
```

CS

```
1  <head>
2      <title>@ViewData["Title"] - Voorbeeld View</title>
3  </head>
```

CS

De titel is zo dynamisch geworden. Dit is interessant om bijvoorbeeld een bepaalde layout toe te passen op meerdere Views.

Html Helpers

HTML-helpers worden gebruikt om HTML-elementen op een dynamische manier te genereren vanuit C#. Hierdoor kun je de hoeveelheid handmatige HTML-code verminderen. Ze binden ook automatisch de gegevens van je model aan de juiste HTML-elementen en ze ondersteunen client-side validatie.

Gebruik:

- Start steeds met de helper klasse `@Html`
- Gevolgd door een extensiemethode

Onderstaand voorbeeld genereert een url: In View:

```
1 @Html.ActionLink("Zie bestelbonnen", "Overzicht", "Bestelbon")
```

In de browser, krijg je onderstaande link in de broncode (CTRL + U, Windows):

```
1 <a href="/Bestelbon/Overzicht">Zie bestelbonnen</a> html
```

Andere voorbeelden van HTML-helpers zijn:

- `@Html.TextBoxFor(model => model.Name)`: genereert een Tekstvak voor een property *Name* uit het model.
- `@Html.DropDownList("Country", new SelectList(new[] { "USA", "Canada" })))`: genereert een dropdownlist met als naam *Country*.
- `@Html.DisplayFor(modelItem => product.ProductID)`: toont de inhoud van de property.
- `@Html.DisplayNameFor(modelItem => product.ProductID)`: in een model kun je een kenmerk [DisplayName] toevoegen aan een property om de naam van de property op een andere manier weer te geven. Deze html-helper toont de DisplayName van die property.

Tag Helpers

Tag-helpers worden gebruikt in HTML-tags om de dynamische HTML-elementen te genereren. In plaats van te beginnen met een `@` krijgt een tag-helper de prefix **asp**.

In onderstaand voorbeeld genereer je een url om de Action Method *Index* uit te voeren van de *BestelbonController*.

```
1 <a asp-controller="Bestelbon" asp-action="Index">Bekijk de bestelbonnen!cs
  </a>
```

In de browser als je broncode zou bekijken (CTRL + U, Windows):

```
1 <a href="/Bestelbon/Overzicht">Zie bestelbonnen</a>html
```

asp-for wordt vaak gebruikt in een formulier om te verwijzen naar bepaalde Properties uit je model

```
1 <form asp-controller="Home" asp-action="Submit" method="post">cs
2     <input asp-for="Name" />
3     <input asp-for="Email" />
4     <button type="submit">Submit</button>
5 </form>
```

Projecten met back-end

Setup

Open het project MVCDemo. Hierin zitten volgende onderdelen:

- Modellen: Klant, Bestelling, Orderlijn en Product
- EntityFramework met databaseconnectie (controleer wel je connectiestring)
- Identity
- UnitOfWork
- GlobalUsings.cs

Gegevens ophalen uit de database

Je hebt al kennis gemaakt met Unit of Work binnen API, deze basiskennis wordt beschouwd als verworven. De werkwijze voor de Controller is gelijklopend met de ApiController. Alleen geef je nu de resultaten terug naar een View in plaats van als JSON.

Maak een `KlantController` en maak een nieuwe actie `Index` voor het ophalen van de klanten. Er wordt gebruik gemaakt van `UnitOfWork`.

```
1 public class KlantController : Controller
2 {
3     private readonly IUnitOfWork _context;
4
5     public KlantController(IUnitOfWork context)
6     {
7         _context = context;
8     }
9
10    public async Task<ActionResult<IEnumerable<Klant>>> Index()
11    {
12        return View(await _context.KlantRepository.GetAllAsync());
13    }
14 }
```

In de View gebruik je Razor en HTML om een overzicht te geven van de klanten. In het onderstaande voorbeeld worden tag- en html-helpers gebruikt. Er zijn ook hyperlinks voorzien voor het aanmaken, wijzigen en verwijderen van klanten.

```
1 @model List<Klant>
2 @{
3     ViewData["Title"] = "Index";
4 }
5 <h2>Klanten</h2>
6 <p> <a asp-action="Create" class="btn btn-primary">Create New</a></p>
7 <table>
8     <thead>
9         <tr>
10             <td>
11                 @Html.DisplayNameFor(v => v[0].Familienaam)
12             </td>
13             <td>
14                 @Html.DisplayNameFor(v => v[0].Voornaam)
15             </td>
16             <td>
17                 @Html.DisplayNameFor(v => v[0].AangemaaktDatum)
18             </td>
19         </tr>
20     </thead>
```

```
21     <tbody>
22         @foreach (var klant in Model)
23         {
24             <tr>
25                 <td>
26                     @Html.DisplayFor(modelitem => klant.Familienaam)
27                 </td>
28                 <td>
29                     @Html.DisplayFor(modelitem => klant.Voornaam)
30                 </td>
31                 <td>
32                     @Html.DisplayFor(modelitem => klant.AangemaaktDatum)
33                 </td>
34                 <td><a asp-action="Edit" asp-route-id="@klant.KlantID"
35 class="btn btn-primary">Edit</a></td>
36                 <td><a asp-action="Details" asp-route-id="@klant.KlantID"
37 class="btn btn-primary">Details</a></td>
38                 <td><a asp-action="Delete" asp-route-id="@klant.KlantID"
39 class="btn btn-primary">Delete</a></td>
40             </tr>
41         }
42     </tbody>
43 </table>
```

[Hier vind je het WebApiDemo-project.](#)

Laatst bijgewerkt: 09-09-2026, 08:52

Bijdragers: JorenSynaeveTM